

State-Space Models and Linear Attention

Their Equivalence, and How the Recurrent Update Is Actually Computed

June 2026

Abstract

Softmax attention costs $O(N^2)$ in sequence length. Two research lines attack this from opposite directions and have now converged: **linear attention** (drop the softmax so the computation can be reassociated into linear time) and **state-space models** (treat the sequence as a linear dynamical system). This report explains why these are, at their core, *the same object* — a linear recurrent network with a matrix-valued state and a per-step decay — and then focuses on the question that actually decides efficiency and accuracy: *how do you compute the recurrent update?* We compare the four methods — recurrence, convolution, the work-efficient **Blelloch parallel scan**, and the modern **chunkwise / semiseparable-matrix factorization** — and show why the field converged on chunking: it re-expresses most of the work as matrix multiplies that hit Tensor Cores, while a pure scan does not.

Contents

1	The setup: escaping quadratic attention	2
2	Linear attention	2
3	State-space models	2
4	The unification: SSM <i>is</i> gated linear attention	3
5	How the update is actually computed	3
5.1	Method 1 — Recurrent (sequential)	4
5.2	Method 2 — Convolution (LTI only)	4
5.3	Method 3 — Parallel associative scan (Blelloch)	4
5.4	Method 4 — Chunkwise / semiseparable factorization	4
5.5	Method 5 — The global factorization $Q(K^\top V)$, and the quadratic $\leftrightarrow$ linear spectrum	5
5.6	Side by side	6
6	Recent advances (a tour)	6
7	The fundamental trade-off, and where the field is	7
8	Fixing the associative-recall gap	7
9	Distilling Transformers into SSMs / linear attention	9
10	Synthesis	10

1 The setup: escaping quadratic attention

TL;DR. Softmax attention is $O(N^2)$ because it forms an $N \times N$ score matrix. Linear attention and SSMs both replace it with a fixed-size recurrent *state*, giving $O(N)$ time and $O(1)$ memory per step — at the cost of bounded recall.

Standard attention computes $O = \text{softmax}(QK^\top)V$, materializing an $N \times N$ matrix: quadratic time and (naively) memory. The alternative is to carry a *summary state* that is updated token by token, so cost grows linearly in N and inference needs only constant memory per step (no growing KV cache). Linear attention and state-space models are two routes to exactly that.

2 Linear attention

Key idea. Drop the softmax and attention becomes *associative*: instead of $(QK^\top)V$ you compute $Q(K^\top V)$. The running $K^\top V$ is a fixed-size matrix state $S_t = \sum_{i \leq t} k_i v_i^\top$, updated by an outer product each step — i.e. a linear RNN.

Replace the softmax similarity with a kernel feature map ϕ , so the (causal) output at t is

$$o_t = \sum_{i \leq t} (\phi(q_t)^\top \phi(k_i)) v_i = \phi(q_t)^\top \underbrace{\sum_{i \leq t} \phi(k_i) v_i^\top}_{S_t} = \phi(q_t)^\top S_t.$$

The crucial move is the middle equality: because there is no softmax wrapping $\phi(q_t)^\top \phi(k_i)$, the sum over i can be *pulled inside*, turning the per-query $O(N)$ sum into a recurrence on a shared state:

$$S_t = S_{t-1} + \phi(k_t) v_t^\top, \quad o_t = \phi(q_t)^\top S_t. \quad (1)$$

This is the “Transformers are RNNs” observation (Katharopoulos et al., 2020): linear attention *is* a recurrent network whose hidden state $S_t \in \mathbb{R}^{d_k \times d_v}$ is the accumulated key–value outer products. Training can still be done in parallel (as a masked matmul), while inference is a cheap constant-memory recurrence.

Why plain linear attention underperformed. Two problems: (1) the state S_t has fixed size $d_k \times d_v$, a *bounded memory* that must summarize the whole past — so recall of specific earlier tokens is weak; and (2) there is no decay, so old information never fades and the state saturates. Both are fixed by the ideas below.

3 State-space models

Key idea. An SSM is a linear dynamical system $h_t = \bar{A}h_{t-1} + \bar{B}u_t$, $y_t = Ch_t$. With *fixed* $\bar{A}, \bar{B}, C$ it is time-invariant and equals a long convolution; making them *input-dependent* (selective) breaks that and forces a scan — which is what Mamba does.

A continuous SSM $x'(t) = Ax(t) + Bu(t)$, $y(t) = Cx(t)$ is discretized to

$$h_t = \bar{A}h_{t-1} + \bar{B}u_t, \quad y_t = Ch_t. \quad (2)$$

S4 (Gu et al., 2021) made this work for long sequences with a structured A (HiPPO initialization for long-range memory, diagonal-plus-low-rank for efficiency), and computed (2) as a *convolution* (below). **S4D/DSS** simplified A to diagonal; **S5** used a parallel *scan* instead of convolution and a MIMO state.

Selectivity (Mamba). The limitation of (2) is that $\bar{A}, \bar{B}, C$ are *constant* across time — the model cannot choose what to remember based on content. **Mamba** (Gu & Dao, 2023) makes $\bar{A}_t, \bar{B}_t, C_t$ functions of the input u_t (“selective”). This input-dependence is what gives the model its power — but it destroys time-invariance, so the convolution view no longer applies and you must use a **parallel scan**. Mamba’s contribution is as much systems as modeling: a hardware-aware selective-scan kernel that runs the recurrence in SRAM and recomputes activations in the backward pass to save memory.

4 The unification: SSM *is* gated linear attention

Key idea. Write the SSM state as a *matrix* and the input term as an outer product: $S_t = \bar{A}_t S_{t-1} + k_t v_t^\top$, $o_t = q_t^\top S_t$. Linear attention is the special case $\bar{A}_t = I$ (no decay); a diagonal/scalar $\bar{A}_t$ is exactly a *gated/decayed* linear attention. Same equation, two literatures.

Put (1) and (2) side by side. Identify the SSM’s input projection $\bar{B}_t u_t$ with the key–value outer product $k_t v_t^\top$, and the readout C_t with the query q_t . Then the SSM recurrence becomes

$$\boxed{S_t = \bar{A}_t S_{t-1} + k_t v_t^\top, \quad o_t = q_t^\top S_t} \quad (3)$$

and the relationship is immediate:

- $\bar{A}_t = I$ (identity): plain **linear attention** (1).
- $\bar{A}_t = \text{diag}(a_t)$ with a fixed decay: **RetNet**’s retention.
- $\bar{A}_t = \text{diag}(a_t)$ with an *input-dependent* gate: **Gated Linear Attention (GLA)**, and (with scalar a_t per head) **Mamba-2**.

So the SSM “state transition matrix” $\bar{A}_t$ is nothing but the **forget gate / decay** applied to the linear-attention KV-state. The two fields were describing the same model in different language: SSMs from control theory, linear attention from kernelized transformers.

The duality made precise (Mamba-2 / SSD). The whole sequence map of (3) can be written as one matrix product $Y = MU$, where M is lower-triangular with

$$M_{ti} = q_t^\top \left(\prod_{i < j \leq t} \bar{A}_j \right) k_i \quad (i \leq t).$$

For scalar/diagonal $\bar{A}$, this M is a **1-semiseparable matrix**: a structured matrix whose below-diagonal blocks are low-rank. Mamba-2’s *State Space Duality* observes that $M = L \odot (QK^\top)$, where $L_{ti} = \prod_{i < j \leq t} a_j$ is a structured **decay mask**. That is exactly *masked attention with a decay mask* — the “quadratic/attention” form — while the recurrence (3) is the “linear/SSM” form. They compute the *same* Y ; which one is faster depends only on how you factor M (next section).

5 How the update is actually computed

TL;DR. Four ways to evaluate the recurrence (3)/(2): **recurrent** (serial, for inference), **convolution** (LTI only), **parallel scan** (Blelloch; handles selectivity), and **chunkwise** (factor the semiseparable matrix into blocks so most work becomes matmuls). Chunkwise wins on modern hardware because it uses Tensor Cores.

This is the heart of the matter: the model in (3) is fixed, but its *computation* can be organized in four ways with very different hardware profiles.

5.1 Method 1 — Recurrent (sequential)

Step through (3) one token at a time. **Work** $O(Nd^2)$, **depth** $O(N)$ — fully serial. Constant memory per step and $O(1)$ work per new token, so it is *ideal for autoregressive inference / decoding*. Useless for training a long sequence: the $O(N)$ depth means no sequence parallelism.

5.2 Method 2 — Convolution (LTI only)

If $\bar{A}, \bar{B}, C$ are *time-invariant*, unrolling (2) gives $y = K * u$, a long convolution with kernel $K = (C\bar{B}, C\bar{A}\bar{B}, C\bar{A}^2\bar{B}, \dots)$. Compute it with an FFT in **work** $O(N \log N)$, **depth** $O(\log N)$. This is how **S4** trains in parallel. The fatal restriction: *it requires time-invariance*, so it cannot be used for selective/gated models (input-dependent $\bar{A}_t$) — which is precisely why Mamba abandoned it.

5.3 Method 3 — Parallel associative scan (Blelloch)

The diagonal recurrence $h_t = a_t \odot h_{t-1} + b_t$ (with $b_t = \bar{B}_t u_t$) is a **first-order linear recurrence**, hence a *scan* over affine maps. Encode each step as (a_t, b_t) and compose with the *associative* operator

$$(a_2, b_2) \bullet (a_1, b_1) = (a_2 a_1, a_2 b_1 + b_2).$$

A work-efficient **Blelloch scan** (up-sweep/down-sweep) then evaluates all states in **work** $O(N)$, **depth** $O(\log N)$ — and, crucially, it does *not* require time-invariance, so it works for selective SSMS. This is the engine of **S5** and **Mamba-1**. The catch is hardware: the scan is *elementwise* (per state channel), so it is **memory-bound** and barely uses the Tensor Cores that dominate a modern GPU’s FLOPs. Mamba-1’s selective scan is a hand-written, memory-bandwidth-optimized CUDA kernel for exactly this reason.

5.4 Method 4 — Chunkwise / semiseparable factorization

The modern workhorse. Split the sequence into chunks of length C and factor the semiseparable matrix M block-wise:

- **Intra-chunk (diagonal blocks):** compute the attention-like *quadratic* form $L \odot (QK^\top)V$ *within* each chunk — a batch of $C \times C$ **matmuls** on Tensor Cores. C is small (e.g. 64–256), so the $O(C^2)$ cost is cheap.
- **Inter-chunk (off-diagonal blocks):** summarize each chunk by its end state S (a $d_k \times d_v$ matrix) and pass it forward with a short *recurrence/scan over only N/C chunk states*; each chunk’s contribution from the past is a low-rank matmul $Q_{\text{chunk}} S_{\text{prev}}$.

Per chunk, concretely. Each token t in chunk c gets *two* contributions — the carried-in state and the local attention:

$$o_t = \underbrace{q_t^\top S_{\text{in}}}_{\text{inter-chunk readout}} + \underbrace{\sum_{i \in c, i \leq t} L_{t,i} (q_t^\top k_i) v_i}_{\text{intra-chunk local attention}}.$$

So a chunk is exactly **three matmuls**: (1) the intra-chunk attention $(L \odot Q_c K_c^\top) V_c$; (2) the inter-chunk readout $Q_c S_{\text{in}}$; and (3) the state update $S_{\text{out}} = \gamma_c S_{\text{in}} + \tilde{K}_c^\top V_c$ (with $\tilde{K}_c$ the keys scaled by their decay to the chunk end, γ_c the chunk’s total decay). Only (3) is recurrent — it chains

$S_{\text{in}} \rightarrow S_{\text{out}}$ across the N/C chunks; everything else is a dense matmul on Tensor Cores. In block-matrix terms this is partitioning the semiseparable M into chunks — diagonal blocks done the quadratic (attention) way, off-diagonal blocks carried by the state:

M (causal, 3 chunks)	how each block is computed
+-----+-----+-----+	
D1 0 0	D_c diagonal block: intra-chunk local attention
+-----+-----+-----+	(L * (Qc Kc^T)) Vc [quadratic]
R D2 0	
+-----+-----+-----+	R off-diagonal: contribution of past chunks,
R R D3	carried by the state -> Qc * S_in [factorized]
+-----+-----+-----+	

Total **work** $O(NCd + Nd^2)$, **depth** $O(N/C + \log C)$, **memory** $O(N)$ — but with the decisive property that the bulk of the FLOPs are **matmuls**, so it runs at Tensor-Core throughput. This chunk-parallel form is used by **RetNet** (parallel/recurrent/chunkwise trio), **GLA**, **Mamba-2 (SSD)**, and **DeltaNet**. It is why Mamba-2 is several times faster than Mamba-1: restricting $\bar{A}_t$ to a *scalar* per head makes the chunk math expressible as matmuls (SSD), trading a little expressivity for Tensor-Core utilization.

5.5 Method 5 — The global factorization $Q(K^\top V)$, and the quadratic $\leftrightarrow$ linear spectrum

The most basic way to make linear attention linear is the associativity move from §2 used as a *standalone, global* computation — form the state once, read it once:

$$KV = K^\top V \in \mathbb{R}^{d_k \times d_v} \quad (\text{one matmul}), \quad O = Q(KV) \quad (\text{one matmul}).$$

Work $O(Nd^2)$, **depth** $O(\log N)$ (the $K^\top V$ reduction), *two dense matmuls*, no scan, no recurrence — the most parallel and most Tensor-Core-friendly option of all. The catch is causality: $KV = \sum_i k_i v_i^\top$ is the *full* sum over *all* positions, so every query sees the whole sequence. That is exactly right for **non-causal** (bidirectional encoder) settings, or for computing a *global* summary — but an autoregressive model needs each q_t to see only its prefix $\sum_{i \leq t}$, which the global factorization does not provide.

Not a rival — a building block. Making the factorization *causal* is precisely what the other methods do: the recurrence and the Blelloch scan compute the *prefix* states $S_t = \sum_{i \leq t} k_i v_i^\top$ (serially, or in $\log N$ depth), and chunkwise uses the global factorization *across chunks* — the inter-chunk readout $Q_c S_{\text{in}}$ is literally $Q(K^\top V)$ with $K^\top V$ accumulated from earlier chunks. So $Q(K^\top V)$ is the engine inside the other methods, exposed directly only when causality is not required.

The spectrum. The chunk size C interpolates between the two pure forms:

- $C = N$ (one chunk): pure **quadratic** attention $(L \odot QK^\top)V$ — $O(N^2d)$ work, all matmul, best when N is *small* (Tensor Cores make N^2 cheap and there is no scan overhead).
- $C = 1$: pure **recurrence / per-step factorization** — $O(Nd^2)$, serial.
- intermediate C : **chunkwise**, choosing the sweet spot.

The quadratic $(QK^\top)V$ and the factorized $Q(K^\top V)$ are the two ends of one axis — dense-but-causal vs. linear-but-prefix-bound — with the quadratic form winning when $N \lesssim d$ and the factorization winning when $N \gg d$. Chunkwise is the engineered middle.

Bidirectional? Use the factorization, not two causal passes. If you want *bidirectional* (non-causal) linear attention, $Q(K^\top V)$ is the right answer. Bidirectional linear attention *is* the single global state $S = \sum_{\text{all } i} \phi(k_i) v_i^\top$ — one reduction and two matmuls compute it directly. Running a forward *and* a backward causal pass instead recomputes per-position prefix *and* suffix states (the costly causal machinery, twice) only to sum them back into that one global state, and double-counts the diagonal $k_t v_t^\top$ (which must then be subtracted). The lone exception is *decayed/gated* attention: position-dependent decay means there is no single global state, so a bidirectional variant genuinely needs two directional passes (or bidirectional chunkwise). Plain bidirectional linear attention, though, is the textbook use case for the factorization.

5.6 Side by side

Method	Work	Depth	Tensor cores	Causal?	Used by
Recurrent	$O(Nd^2)$	$O(N)$	no	yes	inference/decode
Convolution (FFT)	$O(N \log N)$	$O(\log N)$	no	yes (LTI only)	S4
Parallel scan (Blelloch)	$O(Nd^2)$	$O(\log N)$	poorly	yes	S5, Mamba-1
Quadratic $(L \odot QK^\top)V$	$O(N^2d)$	$O(\log N)$	yes	yes	short-seq / intra-chunk
Factorization $Q(K^\top V)$	$O(Nd^2)$	$O(\log N)$	yes	no (global)	encoders / inter-chunk
Chunkwise / SSD	$O(NCd)$	$O(N/C + \log C)$	yes	yes	RetNet, GLA, Mamba-2, DeltaNet

Table 1: Six ways to evaluate the same map $Y = MU$. The *factorization* $Q(K^\top V)$ and the *quadratic* $(QK^\top)V$ are the two extremes (global-parallel vs. dense); chunkwise interpolates. The factorization alone computes the *full / non-causal* state — causal models need the prefix states supplied by the recurrent / scan / chunkwise rows. Convolution adds selectivity-free (LTI) parallelism; the scan restores selectivity but is memory-bound.

The throughline. All of these evaluate one structured matrix transform $Y = MU$. Recurrence and convolution are the two “pure” factorizations (linear vs. spectral); the Blelloch scan is the work-efficient parallel evaluation of the linear form; chunkwise is a *block* factorization that does the small diagonal blocks the quadratic (attention) way and the large off-diagonal structure the linear (scan) way — explicitly interpolating between the attention and SSM forms of the same model.

6 Recent advances (a tour)

TL;DR. The recent models are all instances of (3) differing in three choices: the *decay/gate* $\bar{A}_t$, the *update rule* (plain outer product vs. delta rule), and the *feature map* — plus a chunkwise kernel to train them.

- **RetNet** (2023): fixed exponential decay; introduced the parallel/recurrent/*chunkwise* trio that the field standardized on.

- **GLA — Gated Linear Attention** (2024): *data-dependent* diagonal gates with a hardware-efficient chunkwise kernel; showed selective gating is compatible with Tensor-Core training.
- **Mamba-2 / SSD** (2024): scalar-per-head decay; the semiseparable-matrix duality that connects SSMs and attention and yields the fast chunkwise form.
- **DeltaNet and Gated DeltaNet** (2024): replace the additive update with the *delta rule* $S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$, which *overwrites* stale memory instead of only adding — markedly better associative recall; parallelized chunkwise over sequence length.
- **RWKV** (v5/6/7, 2024–25) and **xLSTM** (mLSTM/sLSTM, 2024): RNN-lineage designs that land on the same gated-linear-recurrence core with matrix states and parallel training.
- **Griffin/Hawk** (2024): gated linear recurrences interleaved with *local* attention — an explicit hybrid.
- **Hybrids** (**Jamba**, **Zamba**, **Based**): interleave a few full softmax-attention layers among many SSM/linear layers to recover exact long-range recall while keeping most layers linear.
- **Test-Time Training layers** (2024): make the state itself a small model updated by a gradient step per token — a more expressive “update rule” that generalizes the outer-product and delta rules.

7 The fundamental trade-off, and where the field is

TL;DR. A fixed-size state buys $O(N)$ time and $O(1)$ memory but caps exact recall — the structural reason linear models trail softmax attention on associative-recall / long-context retrieval. Better update rules (delta), bigger states, and attention hybrids are how the gap is being closed.

Everything above shares one feature: a *bounded* state S_t summarizing an unbounded past. That is the source of the efficiency (constant inference memory, no KV cache) *and* of the accuracy gap: tasks needing exact recall of arbitrary earlier tokens (“needle in a haystack,” associative recall) expose the finite state. The research response is exactly along the three knobs of (3): smarter *decay* (selective gates), smarter *update rules* (delta rule, test-time training) that manage the limited memory rather than just accumulate, larger state dimensions made affordable by chunkwise kernels, and *hybrids* that sprinkle in a few exact-attention layers. The convergence of the SSM and linear-attention literatures around (3) and the chunkwise/semiseparable computation is what made this a single, fast-moving field rather than two.

8 Fixing the associative-recall gap

TL;DR. A fixed state can hold only so many key-value pairs before they interfere — the *associative-recall* gap (measured by the MQAR benchmark). The fixes push the recall-throughput frontier four ways: *sharper/bigger state* (power attention, learned feature maps), *better update rules* (the delta rule), *hybrids* (a few exact-attention layers), and *more expressive memory* (test-time training).

Why recall is the hard case. To retrieve v_j from a query $q_j \approx k_j$, the readout $q_j^\top S = \sum_i (\phi(q_j)^\top \phi(k_i)) v_i$ must satisfy $\phi(q_j)^\top \phi(k_i) \approx \delta_{ij}$ — the feature map has to keep distinct keys *near-orthogonal*. The number of near-orthogonal keys you can pack is bounded by the **effective dimension of the state**, so recall capacity scales with state size, and there is a measured

recall-throughput Pareto frontier (Arora et al.’s *Zoology* / MQAR). Every fix below buys recall by enlarging or sharpening that state, or by spending a little exact attention.

1. Sharper, higher-order feature maps — power attention. Softmax owes its recall to an effectively infinite-dimensional, very “spiky” similarity. Plain linear attention uses a low-degree map, which is too smooth to separate many keys. **Power attention** raises the similarity to a power: use $(q^\top k)^p$ instead of $q^\top k$. By the tensor identity

$$(q^\top k)^p = \langle q^{\otimes p}, k^{\otimes p} \rangle,$$

this is linear attention with the p -fold *tensor-power* feature map $\phi(x) = x^{\otimes p}$, whose dimension is d^p — so the state becomes a $d^p \times d_v$ matrix. The exponent p is a **dial**: $p = 1$ is ordinary linear attention; larger p sharpens the similarity (toward a hard max) and *exponentially* enlarges the state, packing far more near-orthogonal keys — yet it stays *linear in sequence length* N (the state is still a fixed matrix accumulated over N , and the chunkwise/factorized machinery still applies). Power attention thus turns the recall–compute trade into a continuous knob, decoupled from N (symmetric tensor powers keep d^p manageable for small p).

1b. Softmax mimicry — a finite proxy for softmax’s infinite state. Read all the feature-map methods through one lens: **the feature dimension is the effective state size**. Softmax attention corresponds to the *infinite*-dimensional exponential kernel $\exp(q^\top k)$ — effectively an unbounded state, which is why it has near-perfect recall (and a growing KV cache). Linear attention replaces it with a *finite* map ϕ , so its state has dimension $\dim \phi$. “Softmax mimicry” is the program of choosing ϕ so that $\phi(q)^\top \phi(k) \approx \exp(q^\top k)$, and then *enlarging* that finite map to push the effective state — and the recall — toward softmax:

- **Random features (Performer / FAVOR+):** an unbiased estimate of $\exp(q^\top k)$ from m positive random features; the state is $m \times d_v$, so more features = larger effective state = closer to softmax.
- **Taylor / polynomial (Based):** $\exp(q^\top k) \approx 1 + q^\top k + \frac{1}{2}(q^\top k)^2$, a feature map of dimension $1 + d + d^2$; higher order = bigger state.
- **Tensor powers (power attention):** $(q^\top k)^p$ is the degree- p term of that same expansion — p dials the state size d^p .
- **Learned spiky maps (Hedgehog):** train ϕ to reproduce softmax’s *low-entropy, monotone* attention weights directly (distillation) — softmax-like sharpness at whatever feature dimension you can afford.
- **Sketching the big state (PolySketchFormer):** a high-degree polynomial state is d^p -dimensional; randomly *sketch* it down to a tractable size (the RandNLA trick) to keep most of the capacity at a fraction of the cost.

The unifying statement: softmax has an unbounded effective state; every linear/SSM model approximates it with a finite one, and these techniques *expand* that finite state — via more random features, higher polynomial degree, or learned spiky maps — trading compute and memory per token for recall. You are choosing how large a finite proxy for softmax’s infinite state you can afford.

2. Bigger state. The blunt knob: simply widen the state dimension. Mamba-2’s chunkwise kernels made large state dims affordable, and recall improves roughly with state size — at proportional compute/memory.

3. Better update rules — the delta rule. Additive accumulation $S += kv^\top$ never *removes* stale information, so keys interfere and the state saturates. **DeltaNet**’s update $S \leftarrow S(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ instead *overwrites* the value stored at key k_t — an online least-squares / fast-weight step — sharply cutting interference and improving associative recall; **Gated DeltaNet** adds forgetting. This raises recall without enlarging the state, by *managing* it better.

4. Hybrids and local attention. Keep most layers linear but add a few *exact* ones: interleave full softmax-attention layers (**Jamba**, **Zamba**) or pair short *sliding-window* attention with linear global attention (**Based**). The exact component handles precise recall (recent or pinpoint lookups); the linear component carries the bulk cheaply. Based was designed explicitly to sit on the recall-throughput frontier.

5. More expressive memory — test-time training. Make the state itself a small network updated by a gradient step per token (**TTT** layers). A learned nonlinear memory can store and address associations a single linear map cannot — generalizing the additive and delta update rules. **The common thread.** Every fix increases either the *capacity* of the state (power attention, bigger state, TTT) or the *precision* with which it is addressed and maintained (delta rule, learned feature maps), or buys exact recall outright with a little attention (hybrids). Power attention is notable for making the capacity a single tunable exponent that interpolates between linear attention and softmax-like sharpness.

9 Distilling Transformers into SSMs / linear attention

TL;DR. Rather than pretrain an SSM from scratch, *convert* a pretrained Transformer: reuse its projections, then distill by matching the student’s *quadratic dual form* (normalized $\phi\phi^\top$, or the semiseparable M) to the teacher’s softmax attention. That matching materializes the $N \times N$ matrix at *training* time; inference still runs the linear form.

Pretraining a linear/SSM model from scratch is expensive, and a pretrained Transformer already holds the knowledge — so a major practical line *converts* one into the other. The supervision comes at several levels, weak to strong:

- **Weight transfer / init:** reuse the teacher’s W_Q, W_K, W_V, W_O in the student (mapped to the SSM’s B, C), so only the mixing *mechanism* (softmax $\rightarrow$ linear/SSM) is new — a large head start over random init.
- **Logit distillation (KL):** standard output-distribution matching — global but a weak signal for the mixing mechanism itself.
- **Hidden-state matching:** MSE between student and teacher layer outputs; needs only the student’s *linear* form, so it is cheap (no $N \times N$).
- **Mixing-matrix matching:** the strongest signal — and the one that materializes the quadratic form. Compare the student’s explicit mixing matrix to the teacher’s softmax attention: the normalized $\phi(q)\phi(k)^\top$ for a linear-attention student, or the *semiseparable* M (from the SSD duality, §4) for an SSD/Mamba student. This costs $O(N^2)$ at train time (one-time, chunkable), but directly teaches the new layer to mix *like* attention.

Why you *can* match matrices. The enabler is the duality: the student has a quadratic dual form (normalized $\phi\phi^\top$, or the materialized M), so you can write it down and align it with softmax

attention — then *deploy* the linear / scan form. Distillation deliberately uses the student’s expensive form to *learn* and its cheap form to *run*. **Hedgehog** is exactly this for a feature-map student: it learns ϕ by matching softmax’s per-query attention *distributions* (a KL loss over the materialized $N \times N$ weights) — train-time-quadratic, inference-linear.

Recipes. **MOHAWK** (Bick et al., 2024) is the canonical Transformer→Mamba pipeline: a three-stage, local-to-global curriculum — (1) match the SSD student’s mixing matrix M to the teacher’s attention matrix; (2) align hidden states layer by layer; (3) end-to-end logit distillation. **“The Mamba in the Llama”** (Wang et al., 2024) reuses the attention projections to initialize Mamba layers, keeps a few attention layers (a hybrid), and distills with KL — recovering most of the teacher’s quality at a fraction of pretraining cost.

10 Synthesis

Concept	One-line takeaway
SSM = linear attention Duality (SSD)	both are $S_t = \bar{A}_t S_{t-1} + k_t v_t^\top$, $o_t = q_t^\top S_t$; $\bar{A}_t$ is the decay/gate the sequence map $Y = MU$ has an attention (quadratic) and a recurrent (linear) form; M is semiseparable
Convolution Blelloch scan	parallel but time-invariant only — killed by selectivity work-efficient, selectivity-friendly, but memory-bound (no Tensor Cores)
Chunkwise / SSD Recall gap	block-factor $M \Rightarrow$ matmuls on Tensor Cores — the current default fixed state caps exact recall; power attention, delta rules, hybrids close it

Bottom line. State-space models and linear attention are two names for a linear RNN with a matrix state and a per-step decay. The interesting engineering is not the model but the *evaluation*: a pure parallel scan (Blelloch) makes the selective recurrence parallel and work-efficient, but it is elementwise and memory-bound; factoring the underlying semiseparable matrix into chunks turns most of the computation back into Tensor-Core matmuls, which is why chunkwise/SSD is how these models are trained today. The remaining frontier is the bounded-state recall gap, attacked with better update rules and attention hybrids.

Selected references. Katharopoulos et al., *Transformers are RNNs*, 2020; Choromanski et al., *Performer*, 2021; Gu, Goel, Ré, *S4*, 2022; Smith et al., *S5*, 2023; Sun et al., *Retentive Network*, 2023; Gu & Dao, *Mamba*, 2023; Yang et al., *Gated Linear Attention*, 2024; Dao & Gu, *Mamba-2 / State Space Duality*, 2024; Yang et al., *Parallelizing Linear Transformers with the Delta Rule*, 2024; Beck et al., *xLSTM*, 2024; De et al., *Griffin/Hawk*, 2024; Lieber et al., *Jamba*, 2024; Sun et al., *Test-Time Training layers*, 2024; Arora et al., *Zoology: Measuring and Improving Recall in Efficient Language Models*, 2023, and *Based*, 2024; Zhang et al., *Hedgehog*, 2024; Manifest AI, *Symmetric Power Transformers / Power Attention*, 2024–2025; Kacham et al., *PolySketchFormer: Fast Transformers via Sketching Polynomial Kernels*, 2024; Bick et al., *Transformers to SSMs: Distilling Quadratic Knowledge to Subquadratic Models (MOHAWK)*, 2024; Wang et al., *The Mamba in the Llama: Distilling and Accelerating Hybrid Models*, 2024.